

Java Viva Preparation

Complete answers to 100 questions: Fundamentals, Control Statements, Arrays and Strings, OOP, Access Modifiers and Packages, Exception Handling

A. Java Fundamentals

1. What is Java?

Java is a high-level, object-oriented, class-based programming language created by **James Gosling at Sun Microsystems (1995)** and now owned by Oracle. It follows the principle "Write Once, Run Anywhere."

2. What are the main features of Java?

Simple, object-oriented, platform independent, secure, robust (strong memory management and exception handling), multithreaded, portable, high performance (JIT compiler), distributed, and dynamic.

3. Why is Java called platform independent?

The compiler converts source code into **bytecode**, not machine code. Bytecode runs on any OS that has a JVM, so the same **.class** file works on Windows, Linux, or macOS. (The JVM itself is platform dependent. Java programs are not.)

4. Difference between source code, bytecode, and machine code?

- **Source code**: human-readable **.java** file.
- **Bytecode**: intermediate, platform-neutral **.class** file produced by **javac**.
- **Machine code**: binary instructions specific to a CPU, produced by the JVM (interpreter/JIT) at runtime.

5. What is the JVM?

The Java Virtual Machine is an abstract machine that loads, verifies, and executes bytecode. It includes the class loader, bytecode verifier, interpreter, JIT compiler, and garbage collector.

6. What is the JDK?

The Java Development Kit is the full package for developing Java applications. It contains the JRE plus development tools such as **javac**, **jar**, **javadoc**, and the debugger.

7. What is the JRE?

The Java Runtime Environment provides what is needed to **run** Java programs: the JVM plus core libraries (e.g., **java.lang**, **java.util**). It has no compiler.

8. Difference between JDK, JRE, and JVM?

JDK = JRE + development tools. JRE = JVM + libraries. JVM = the engine that executes bytecode.
Nesting: the JDK contains the JRE, which contains the JVM.

9. What is bytecode?

Bytecode is the intermediate, platform-independent instruction set stored in **.class** files, generated by the Java compiler and executed by the JVM.

10. Why is Java considered an object-oriented language?

Programs are organized around classes and objects, and Java supports encapsulation, inheritance, polymorphism, and abstraction. (It is not 100% pure OO, because primitive types are not objects.)

11. What is the role of the main() method?

It is the entry point of a Java program. The JVM starts execution from it.

12. Why is main() declared public static void?

- **public**: the JVM must access it from outside the class.
- **static**: the JVM calls it without creating an object.
- **void**: it returns nothing to the JVM.
- **main**: the fixed name the JVM looks for.
- **String[] args**: holds command-line arguments.

13. Can we execute a Java program without main()?

Not in standard Java: you get an error such as "main method not found". Before Java 7, a **static** block could print output and then fail. Newer Java versions (preview "unnamed classes" in Java 21+) relax the exact **main** signature, but an entry-point method is still needed.

14. What is a Java class?

A blueprint or template that defines the data (fields) and behavior (methods) of objects.

15. What is an object?

An instance of a class, with **state** (field values), **behavior** (methods), and **identity**. Created using **new**.

16. Difference between a class and an object?

A class is a logical template and does not hold instance data by itself. An object is the real entity created from it and occupies heap memory. One class can have many objects.

17. What is a reference variable?

A variable that holds the memory address (reference) of an object, not the object itself. Example: **Student s = new Student();** where **s** is the reference.

18. Primitive vs reference data types?

Primitives store actual values, have fixed sizes, and are not objects (**int**, **char**). Reference types store addresses of objects on the heap (**String**, arrays, classes) and can be **null**.

19. The eight primitive data types?

byte (1 byte), **short** (2), **int** (4), **long** (8), **float** (4), **double** (8), **char** (2), **boolean** (true/false).

20. What is type casting?

Converting a value from one data type to another.

- **Widening (implicit)**: smaller to larger, automatic (**int** to **double**).
- **Narrowing (explicit)**: larger to smaller, done manually, may lose data: **int x = (int) 9.7;** gives 9.

B. Variables, Operators and Control Statements

21. What is a local variable?

Declared inside a method, constructor, or block. It is created when the block runs and destroyed when it ends. It has no default value (must be initialized) and cannot use access modifiers.

22. What is an instance variable?

Declared inside a class but outside methods, without **static**. Each object gets its own copy. It has default values (0, false, null).

23. What is a static variable?

Declared with **static**. There is only one copy, shared by all objects of the class. It is loaded when the class is loaded.

24. Difference between == and =?

= is the assignment operator (**x = 5**). **==** is the equality comparison operator (**x == 5**) and returns a boolean.

25. Difference between == and .equals()?

== compares values for primitives and **references (addresses)** for objects. **.equals()** compares **content** when overridden (as in **String**).

```
String a = new String("hi"), b = new String("hi");
a == b;           // false
a.equals(b);      // true
```

26. What are relational operators?

==, !=, >, <, >=, <=. They compare two values and return a boolean.

27. What are logical operators?

&& (AND), **||** (OR), **!** (NOT). They combine or negate boolean expressions.

28. Difference between && and &?

&& is short-circuit: if the left side is false, the right side is not evaluated. **&** always evaluates both sides, and also works as a bitwise AND on integers.

29. What is the ternary operator?

A shorthand if-else: **condition ? valueIfTrue : valueIfFalse;**

Example: **int max = (a > b) ? a : b;**

30. Difference between break and continue?

break exits the loop or switch completely. **continue** skips the rest of the current iteration and moves to the next one.

31. Difference between while and do-while?

while checks the condition first (entry-controlled), so it may run zero times. **do-while** runs the body first and checks afterward (exit-controlled), so it runs at least once.

32. When would you use a for loop?

When the number of iterations is known in advance (counting, iterating over arrays): **for(init; condition; update)**.

33. What is a nested loop?

A loop inside another loop. The inner loop completes fully for each iteration of the outer loop. Used for patterns, matrices, and tables.

34. What is a switch statement?

A multi-way branch that compares one variable against several constant **case** values, with an optional **default**. It is an alternative to long if-else-if ladders.

35. Can a switch statement work with a String?

Yes, since Java 7. It also works with **byte**, **short**, **int**, **char**, enums, and wrapper types, but not **long**, **float**, **double**, or **boolean**.

36. What happens if break is omitted from a switch case?

Fall-through: execution continues into the following cases until a **break** or the end of the switch is reached.

C. Arrays and Strings

37. What is an array?

A container that holds a fixed number of elements of the same type, stored in contiguous memory and accessed by index (starting at 0).

38. Why are arrays called fixed-size data structures?

The size is set when the array is created and cannot change. To resize, you must create a new array and copy the elements.

39. How do you declare and initialize an array?

```
int[] a = new int[5];           // size 5
int[] b = {1, 2, 3};           // with values
int[] c = new int[]{4, 5, 6};
```

40. Default value of elements in an integer array?

0. (Also **false** for boolean, **null** for objects, **0.0** for double.)

41. How do you find the length of an array?

Use the **length** property (no parentheses): **arr.length**.

42. What happens when you access an invalid array index?

It throws **ArrayIndexOutOfBoundsException** at runtime. Valid indexes are 0 to **length-1**.

43. What is a two-dimensional array?

An array of arrays, like a table or matrix: **int[][] m = new int[3][4];** (3 rows, 4 columns). Rows can have different lengths (jagged array).

44. Difference between an array and an ArrayList?

Array	ArrayList
Fixed size	Dynamic size
Holds primitives and objects	Holds objects only (wrapper types)
Uses length	Uses size()
Faster, less overhead	More flexible, built-in methods (add , remove)

45. What is a String in Java?

A sequence of characters represented by the **java.lang.String** class (an object, not a primitive). String literals are stored in the String Constant Pool.

46. Why is String immutable?

Once created, its content cannot change. Reasons: **security** (used for passwords, file paths, class names), **thread safety**, **String pool sharing** (saves memory), and **hashcode caching** (good for HashMap keys).

47. Difference between String, StringBuilder, and StringBuffer?

- **String**: immutable.
- **StringBuilder**: mutable, not thread-safe, fastest.
- **StringBuffer**: mutable, thread-safe (synchronized), slower.

48. Difference between equals() and equalsIgnoreCase()?

equals() is case-sensitive: **"Java".equals("java")** is false. **equalsIgnoreCase()** ignores case: it returns true.

49. What is the use of length() in a String?

Returns the number of characters in the string: **"Hello".length()** is 5.

50. What is the use of substring()?

Extracts part of a string. **substring(begin)** goes from **begin** to the end. **substring(begin, end)** goes from **begin** to **end-1**. Example: **"Hello".substring(1,4)** gives **"ell"**.

51. What is the use of `charAt()`?

Returns the character at a given index: `"Java".charAt(2)` gives `'v'`. Throws `StringIndexOutOfBoundsException` if the index is invalid.

52. Difference between `trim()` and `strip()`?

Both remove leading and trailing whitespace. `trim()` removes only characters up to U+0020 (basic ASCII spaces). `strip()` (Java 11+) is Unicode-aware and removes all Unicode whitespace. There are also `stripLeading()` and `stripTrailing()`.

53. What do `toUpperCase()` and `toLowerCase()` do?

They return a **new** string with all characters converted to upper or lower case. The original string is unchanged (immutability).

54. What is the use of `contains()` in `String`?

Checks whether a string contains a given sequence of characters and returns a boolean: `"Hello World".contains("World")` is true.

55. Difference between `concat()` and the `+` operator?

`concat()` accepts only a `String` and throws `NullPointerException` for `null`. `+` accepts any type (auto-converts, so `"a" + 5` works) and treats `null` as the text `"null"`. `+` is more flexible.

D. OOP Concepts

56. What are the four main principles of OOP?

Encapsulation, Inheritance, Polymorphism, and Abstraction.

57. What is encapsulation?

Wrapping data (variables) and the methods that operate on it into a single unit (class), and restricting direct access to the data. It is also called data hiding.

58. How is encapsulation achieved in Java?

Declare variables `private` and provide `public` getters and setters.

```
class Student {
    private int age;
    public int getAge() { return age; }
    public void setAge(int a) { if (a > 0) age = a; }
}
```

59. What is inheritance?

A mechanism where one class (child/subclass) acquires the properties and methods of another (parent/superclass) using `extends`. It models an **IS-A** relationship.

60. Why is inheritance used?

Code reusability, method overriding (runtime polymorphism), and a logical class hierarchy.

61. Difference between `extends` and `implements`?

`extends`: a class inherits a class (or an interface extends an interface). Classes support only single inheritance. `implements`: a class implements one or more interfaces and must define all of their abstract methods.

62. What is polymorphism?

"Many forms": the same method name behaving differently depending on context. It has two types: compile-time and runtime.

63. What is compile-time polymorphism?

The method call is resolved at compile time (static binding). Achieved through **method overloading**.

64. What is runtime polymorphism?

The method call is resolved at runtime (dynamic binding) based on the actual object type. Achieved through **method overriding** with upcasting.

```
Animal a = new Dog();  
a.sound();    // calls Dog's version
```

65. What is method overloading?

Multiple methods in the same class with the same name but different parameter lists (number, type, or order of parameters).

66. What is method overriding?

A subclass provides its own implementation of a method already defined in its superclass, with the same name, same parameters, and a compatible return type.

67. Difference between method overloading and overriding?

Overloading	Overriding
Same class	Parent and child classes
Parameters must differ	Parameters must be the same
Compile-time polymorphism	Runtime polymorphism
Return type can vary	Return type same or covariant
Can be static or private	Cannot override static, private, or final methods

68. Can we overload a method by changing only its return type?

No. It causes a compile-time error, because the compiler cannot decide which method to call from the call alone.

69. What is the super keyword?

Refers to the immediate parent class.

- `super.method()` calls the parent's method.
- `super.variable` accesses the parent's field.
- `super()` calls the parent constructor (must be the first statement).

70. What is the this keyword?

Refers to the current object.

- Distinguish instance variables from parameters (`this.name = name`).
- Call another constructor of the same class (`this()`).
- Pass or return the current object.

71. What is a constructor?

A special block that initializes an object when it is created. It has the same name as the class and no return type. If you write none, the compiler provides a default constructor.

72. Difference between a constructor and a method?

A constructor has the class name, no return type, is called automatically with `new`, and initializes objects. A method has any name, has a return type, is called explicitly, and performs actions.

73. Can a constructor be overloaded?

Yes. Multiple constructors with different parameter lists are allowed.

74. Can a constructor be inherited?

No. Constructors are not inherited, but a subclass constructor can call the parent's constructor using `super()`.

75. What is an abstract class?

A class declared with **abstract** that cannot be instantiated. It can have abstract methods (no body) and concrete methods, plus constructors and fields. Subclasses must implement the abstract methods.

76. What is an interface?

A contract that declares methods a class must implement. Methods are **public abstract** by default, and fields are **public static final**. Since Java 8 it can also have **default** and **static** methods (and **private** ones since Java 9).

77. Why do we use interfaces?

To achieve full abstraction (traditionally), multiple inheritance of type, loose coupling, and a common standard for unrelated classes.

78. Difference between an abstract class and an interface?

Abstract class	Interface
Uses extends	Uses implements
Single inheritance only	Multiple allowed
Can have constructors and instance variables	No constructors; variables are constants
Any access modifier on methods	Methods are public
Partial abstraction	Full abstraction (plus default methods)

79. Can a class implement multiple interfaces?

Yes: **class A implements I1, I2 { }**. This is how Java achieves multiple inheritance.

80. Can an interface extend another interface?

Yes, and it can extend several: **interface C extends A, B { }**.

E. Access Modifiers, Packages and Core Concepts

81. What are the access modifiers in Java?

private, default (no keyword), **protected**, and **public**.

82. Difference between private, default, protected, and public?

Modifier	Same class	Same package	Subclass (other package)	Anywhere
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

83. Why are data members generally declared private?

For encapsulation: it protects data from unauthorized change, allows validation in setters, and lets you change the internal implementation without affecting other code.

84. What is a package?

A namespace that groups related classes, interfaces, and sub-packages, like a folder. Example: **java.util**.

85. Why are packages used in Java?

Organizing code, avoiding name conflicts, access protection, easier maintenance, and reusability.

86. Difference between import and package?

package declares which package the current class belongs to (the first statement in the file). **import** brings classes from other packages into scope so you can use them without writing the full name.

87. What is the purpose of the static keyword?

Makes a member belong to the **class**, not to objects. It applies to variables, methods, blocks, and nested classes. Static members are accessed via the class name and save memory. Static methods cannot use **this** or directly access non-static members.

88. What is the purpose of the final keyword?

Restricts modification.

- A **final variable** is a constant.
- A **final method** cannot be overridden.
- A **final class** cannot be inherited (e.g., **String**).

89. Can a final variable be modified?

No, it cannot be reassigned once initialized. For a final reference, the reference cannot change, but the object's internal state can (e.g., adding items to a final list).

90. Can a final method be overridden?

No. It can be inherited and called, but not overridden.

F. Exception Handling

91. What is an exception?

An unwanted event during program execution that disrupts the normal flow of instructions, such as dividing by zero or accessing a missing file. It is an object of a subclass of **Throwable**.

92. What is exception handling?

A mechanism to detect and respond to runtime problems using **try**, **catch**, **finally**, **throw**, and **throws**, so the program can continue or terminate gracefully.

93. Difference between an error and an exception?

Errors (**OutOfMemoryError**, **StackOverflowError**) are serious system-level problems that applications should not try to catch. Exceptions are conditions the program can and should handle. Both extend **Throwable**.

94. Difference between checked and unchecked exceptions?

- **Checked**: verified at compile time; must be handled or declared (**IOException**, **SQLException**). They extend **Exception** but not **RuntimeException**.
- **Unchecked**: occur at runtime; handling is optional (**NullPointerException**, **ArithmeticException**, **ArrayIndexOutOfBoundsException**). They extend **RuntimeException**.

95. What is the purpose of try and catch?

try holds code that may throw an exception. **catch** holds the code that handles a specific exception type.

```
try { int x = 10 / 0; }
catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

96. What is the purpose of the finally block?

It always executes, whether or not an exception occurs, and is used for cleanup (closing files, connections). It does not run only if the JVM exits (**System.exit()**) or crashes.

97. Difference between throw and throws?

throw explicitly throws an exception object inside a method body: `throw new Exception("msg");`. **throws** is used in the method signature to declare the exceptions it might pass to the caller: `void read() throws IOException`.

98. Can we create our own exception class?

Yes (a custom exception). Extend **Exception** (checked) or **RuntimeException** (unchecked).

```
class AgeException extends Exception {  
    AgeException(String msg) { super(msg); }  
}
```

99. What happens if an exception is not handled?

It propagates up the call stack. If nobody handles it, the JVM's default handler prints the exception name, message, and stack trace, and the program terminates abnormally.

100. Why should exceptions be handled instead of letting the program terminate unexpectedly?

To keep the program running smoothly, give users meaningful messages, avoid data loss or corruption, release resources properly, separate error-handling code from normal logic, and make the application more robust and reliable.